OpenVAF-r Compile-Time Analysis

Where compilation time goes, and which levers actually move it

Ngspice + OpenVAF Enhancements project

July 2026

Contents

OpenVAF-r compile-time analysis

A profiling deep-dive into how long `openvaf-r` takes to compile a Verilog-A model to a `.osdi` library, where that time goes, and which levers actually move it. The short version: compilation is dominated by LLVM optimizing one large function, the build already parallelizes what it can, and the only real knob is the optimization level — which is a compile-time-versus-simulation-speed trade-off, not a free win.

This is a companion to the OpenVAF compiler internals guide (which explains the pipeline) and the robustness campaign.

Baselines

Best-of-two compile time (`openvaf -r <model>.va -o <model>.osdi`, default `-O3`), on an Apple-silicon machine, across the production compact-model corpus:

model	compile time
ekv	0.17 s
vbic	0.27 s
psp103	0.38 s
mextram	0.46 s
hicum2	0.47 s
diode_cmc	0.79 s
bsim6	1.58 s
bsim4	2.22 s
bsimcmg	3.73 s

Compile time tracks the *total* code size (a model's top file plus its includes), not the top file's line count — a 38-line `bsimcmg` top file that `` `includes `` a large model is the slowest of all. The big CMC MOSFET models (bsimcmg, bsim4, bsim6) dominate; everything smaller compiles in well under a second.

Where the time goes

Profiling `bsimcmg` (the slowest) with the macOS `sample` profiler and bucketing stack frames by compiler crate / phase:

phase	share
LLVM (optimize + emit the per-module machine code)	~70 %
`hir_lower` (HIR → MIR lowering)	~16 %
`mir_build` (SSA construction)	~9 %
`hir_ty` (type inference)	~2 %
`hir_def` (item-tree / desugaring)	~2 %
everything else (`mir_opt`, `osdi`, `salsa`, …)	~1 %

So roughly 70 % LLVM back-end, ~25 % front-end (lowering + SSA), ~5 % other. The MIR-level optimizer (`mir_opt`) is a rounding error — almost all optimization happens in LLVM.

Cross-checking against the optimization level tells the same story: at `-O0` (LLVM does minimal optimization but still runs instruction selection and register allocation) `bsim4` compiles in 1.34 s versus 2.22 s at `-O3`, so LLVM's optimization *passes* alone are ~0.9 s — about 40 % of the total — on top of the unavoidable codegen and front-end.

It is already parallel — but bound by one function

The OSDI backend does not compile the model as a single unit. For each module it spawns four independent LLVM tasks onto a `rayon` thread pool — `access`, `setup_model`, `setup_instance`, and `eval` — each building and optimizing its own LLVM module. So the LLVM work is already parallelized across those functions.

The catch is that **`eval`** — the device evaluation, with all the physics and the autodiff-generated Jacobian — dwarfs the other three. Measuring CPU time against wall time:

model	wall	CPU (user+sys)	effective parallelism
bsim4	2.26 s	3.83 s	1.7×
bsimcmg	3.80 s	6.69 s	1.8×

On a 24-core machine, the compile only reaches ~1.7–1.8× parallelism: the three small tasks finish quickly and then 22 cores sit idle while a single thread grinds through optimizing the one big `eval` module. That monolithic `eval` module is the serial critical path.

The only real lever: the optimization level

`openvaf-r` exposes `-O {0,1,2,3}` (default 3). Measuring both compile time and simulation runtime (the latter on a 200-MOSFET transient, 2015 timesteps, where the model's `eval` dominates the simulator's work):

level	compile vs O3	simulation runtime vs O3
O0	−40 %	+50 %
O1	−20 % (bsim4) … −31 % (bsimcmg)	+0.3 % (bsim4), +3.3 % (bsimcmg)
O2	~0 % (same as O3)	≈ O3
O3 (default)	—	baseline

Two things stand out:

- **`-O2` and `-O3` compile in the same time.** The extra passes O3 enables over O2 are cheap; nothing is lost, compile-time-wise, by defaulting to O3.
- The aggressive O2/O3 optimizations buy almost nothing in simulation speed for device-eval code. That code is straight-line, scalar, transcendental-heavy math with tight data dependencies — exactly the shape that resists the vectorization and aggressive unrolling O2/O3 add. `-O1` (mem2reg, inlining, instcombine, GVN, DCE — the optimizations that *do* matter here) captures essentially all of the runtime benefit: within 0.3 % on bsim4, 3.3 % on bsimcmg. Only `-O0`, which skips those core cleanups, pays a real runtime penalty (+50 %).

A tempting idea — disabling the SLP vectorizer (≈18 % of LLVM time on the big models, and prominent in the profile) — does not work: the `default<O3>` textual pass pipeline hard-codes the vectorizers and ignores the `LLVMPassBuilderOptionsSetSLPVectorization` flag (a build with it set to `false` still spent 17.5 % of LLVM time in SLP). Removing those passes would require constructing a custom pass pipeline, and — per the runtime numbers above — would save compile time at little cost, but the win is modest and the change is fragile.

Conclusion

There is no large ”free” compile-time win available. Compilation is fundamentally the cost of LLVM optimizing the model's `eval` function, and the build already parallelizes everything except that one serial module. The available knob is the optimization level, which is a genuine trade-off:

- Production / shipping models: keep **`-O3`** (the default). A model is compiled once and simulated for the lifetime of the design, so simulation speed dominates; and O3 costs no more compile time than O2. This is the chosen policy — the default is unchanged.
- Iterative model development: **`-O1`** is available. When you recompile constantly and simulation speed does not matter, `-O1` cuts compile time 20–31 % at a negligible-to-small (0–3 %) runtime cost.

Future work: splitting the **`eval`** module — and why it isn't a clean win

The obvious idea for reclaiming the idle cores is to split the monolithic **`eval`** module so its LLVM optimization parallelizes. On a large model that could approach a 2× wall-clock reduction. But it is a significant change to the OSDI codegen, and — as the numbers show — it is *not* a free win. It has a cost at both ends.

Small models: the fixed-cost floor makes it a net loss. A trivial resistor (whose `eval` is a single divide) compiles in 0.094 s at **`-O3`** versus 0.090 s at **`-O0`** — optimization adds only ~4 ms, so essentially the entire ~90 ms is fixed overhead: process startup, the front-end, and the *setup / build / emit / link* of the modules that already exist. Splitting `eval` there would spread work that is already sub-millisecond while adding more of exactly that fixed overhead — a new `LLVMContext` and module, another object file to emit and link (the final `.osdi` links every `.o`, so more of them means more linker work), and a thread spawn/join. Below some size threshold the compile is dominated by that floor, so extra modules can only make it slower.

Large models: it trades compile time for simulation speed. The existing four-way split (`access` / `setup_model` / `setup_instance` / `eval`) is free parallelism because those are *genuinely independent* OSDI entry-point functions — they never inline into one another, so nothing is lost. `eval`, by contrast, is one logical function. Parallelizing it means breaking it into separate functions in separate modules, which introduces call boundaries and loses cross-function inlining and optimization across the split (openvaf builds with LTO off). For the device hot path that runs millions of times in a simulation, that is a *runtime* cost — the very thing the `-O3` default exists to protect.

So a real implementation would have to be adaptive, not unconditional: split `eval` only when it is large enough that the parallel compile savings clearly beat the fixed per-split overhead, keep the monolithic module below that threshold (leaving small models untouched), and choose split points that minimize the inlining loss — the same shape as `rustc`'s codegen-units or ThinLTO import thresholds. The net is a plausible win only on the handful of big CMC models, and even there a compile-vs- runtime trade — which is why it stays future work rather than an obvious optimization.

Reproducing

- Phase breakdown: run a compile of a large model (`bsimcmg`, `bsim4`) and sample it with macOS `sample <pid>`, then bucket frames by crate prefix (`llvm`, `hir_lower`, `mir_build`, …).
- Parallelism: compare wall time to CPU time (`/usr/bin/time -l`); CPU ≫ wall means parallel, CPU ≈ wall means serial.
- Opt-level trade-off: compile the same model at `-O0/1/2/3`, time each, then build a `.osdi` at each level and time a device-heavy transient in ngspice (many instances × many timesteps) so the model `eval` dominates the simulator's runtime.
- Fixed-cost floor: compile a trivial model (a one-line resistor) at `-O3` and at `-O0`; their near-equal times (≈90 ms here, differing by a few ms) are almost entirely startup + front-end + module setup/emit/link, with negligible optimization — the overhead any further module split would add to on top.